

ArmorIQ Policy Isolation

Research & Strategy Report

Moving ArmorIQ's policy enforcement to a trust boundary that no AI agent can breach.

Covers: problem analysis, competitive landscape, architecture, security deep dive, implementation strategy, and next steps.

Prepared by: ArmorIQ Engineering
Date: May 2025
Status: Brainstorming / Internal Review
Version: 1.0

Table of Contents

Part 1	Why We Need This	3
1.1	The Fundamental Security Problem	3
1.2	What We've Built So Far (Current State)	3
1.3	What's Actually Secure vs Security Theater	3
1.4	Why This Matters Beyond ArmorClaude	4
Part 2	What the Industry Does	5
2.1	Competitive Architecture Comparison	5
2.2	Industry Patterns That Work	5
2.3	The Key Insight From Industry	6
Part 3	What Will Work, What Won't	7
3.1	Approaches That WILL Work	7
3.2	Approaches That WON'T Work	8
Part 4	Detailed Architecture	9
4.1	The ArmorIQ Policy Daemon	9
4.2	How Policy Mutation Works (Secure Flow)	10
4.3	How Enforcement Works (Fast Path)	11
4.4	ArmorClaude After Migration	11
4.5	Cross-Product Architecture	12
Part 5	Security Deep Dive	13
5.1	Attack Vector Analysis (Current vs Daemon)	13
5.2	The Socket Authentication Model	14
5.3	Failure Modes	15
5.4	Comparison: ArmorIQ vs Industry	15
Part 6	Implementation Strategy	16
6.1	Phase Overview	16
6.2	What Goes Where	16
6.3	Installer Experience	17
Part 7	Open Questions for Brainstorming	18
Part 8	Risk Assessment	19

Part 1: Why We Need This

1.1 The Fundamental Security Problem

ArmorClaude enforces policies on AI tool usage inside Claude Code. Today, the enforcement layer and the AI agent run as the same OS user on the same machine. This means:

- Every file ArmorClaude can read/write, Claude can too
- Every process ArmorClaude can spawn, Claude can too
- Every socket ArmorClaude can connect to, Claude can too
- Every credential ArmorClaude can access, Claude can too

Analogy: It's like hiring a security guard who shares the same keycard as the people they're guarding. The guard can check badges, but anyone being checked can also grab the guard's clipboard and rewrite the access list.

1.2 What We've Built So Far (Current State)

Over 7 phases, we built a comprehensive policy system with 195 automated tests:

Layer	What it does	Security strength
UserPromptSubmit hook + blockPrompt()	Human-only policy commands (/armor-policy)	Strong -- architectural guarantee from Claude Code. LLM cannot forge this event.
Path guard (PreToolUse)	Blocks Write/Edit/Bash writes to policy files	Medium -- catches obvious attempts, bypassable with obfuscation
Handler name guard	Blocks Bash commands referencing handleArmorPolicyCommand etc.	Medium -- catches direct invocation, bypassable with encoding
Crypto digest (CSRG)	Detects policy.json tampering after the fact	Strong -- cryptographic, but reactive not preventive
MCP deny-by-default	Unknown MCP servers blocked until approved	Strong -- gate in PreToolUse, human approval via hook
OPA enforcement mode	Cloud policy evaluation with circuit breaker	Strong -- but only when cloud is reachable

1.3 What's Actually Secure vs What's Security Theater

Actually secure (architectural/cryptographic guarantees):

- UserPromptSubmit hook -- Claude Code guarantees this fires only on human input
- blockPrompt() -- consumes input before LLM sees it
- CSRG crypto digest -- detects tampering mathematically
- policy_update MCP tool deletion -- tool literally doesn't exist

Security theater (best-effort, bypassable by determined LLM):

- Path guard regex -- Claude can obfuscate (base64 -d | node, string concatenation)
- Handler name blocking -- Claude can import via dynamic import() with constructed string

- File permission checks -- same OS user, Claude can chmod
- Any in-process guard -- Claude has the same capabilities as the guard

1.4 Why This Matters Beyond ArmorClaude

ArmorIQ has multiple AI tool products:

- ArmorClaude -- Claude Code enforcement
- ArmorCopilot -- GitHub Copilot enforcement (planned)
- ArmorCodex -- OpenAI Codex enforcement (planned)
- Custom agents -- via ArmorIQ SDK

A shared, isolated enforcement layer would:

1. Eliminate duplicate enforcement code across products
2. Provide consistent policy across all AI tools a developer uses
3. Centralize credential management (one API key location, protected)
4. Enable org-wide policy management from a single control plane

Part 2: What the Industry Does

2.1 Competitive Architecture Comparison

Product	Enforcement Location	How Agent Can't Modify Policy	Latency	License
Invariant Guardrails	Gateway proxy (separate process)	Agent's URLs rewrite to proxy; policies in gateway	Near-zero (pipelined)	Open source
Lasso Security	MCP Gateway proxy	External gateway intercepts all MCP tool calls	Real-time per tool call	Commercial (gateway OSS)
Lakera Guard	Cloud API / self-hosted	API-based; agent sends payloads to external service	Sub-50ms API call	Commercial (Cisco)
Arthur AI Shield	Federated control/data plane	Control plane in cloud; data plane in VPC	p95 <200ms	Commercial (engine OSS)
OPA / Styra	Sidecar PDP	Sidecar has own filesystem; policies from bundle server	Sub-millisecond	Open source (Apache 2.0)
Permit.io / OPAL	Sidecar PDP (OPA + OPAL)	OPAL server pushes diffs via pub/sub	1-5ms local eval	OPAL OSS; Permit.io commercial
Cerbos	Sidecar / service / Lambda	Stateless PDP loads from git/S3	Sub-millisecond	Open source (Apache 2.0)
Anthropic built-in	In-model (training-time)	Weights are immutable at inference	Zero	Built into Claude
ArmorClaude (current)	Same-process hooks	Best-effort guards + crypto detection	<5ms (daemon)	Proprietary

2.2 Industry Patterns That Work

Pattern 1: Gateway/Proxy (Invariant, Lasso)

- Agent literally cannot reach tools without passing through the enforcement layer
- URLs are rewritten, agent never has direct access
- Why it works: Physical interposition. Agent can't skip what it can't see around.
- Relevance to us: Claude Code's hook system IS a form of interposition -- PreToolUse fires before every tool call. But the hooks run as the same user, so the isolation is logical, not physical.

Pattern 2: Sidecar PDP (OPA, Cerbos, Permit.io)

- Separate process, separate filesystem, policies pushed from external authority
- Application calls check(action, resource), PDP returns allow/deny
- Why it works: Policy authoring and policy evaluation are in separate trust domains. App can query but never modify.
- Relevance to us: This is exactly what the ArmorIQ daemon should be -- a local PDP that the AI agent can query but not control.

Pattern 3: Cloud API (Lakera, Arthur)

- External service makes decisions, agent has no local state to tamper with

- Why it works: Nothing local to attack.
- Why it DOESN'T fully work for us: 50-200ms latency per decision is too slow for CLI tools. Also requires internet.

Pattern 4: In-model alignment (Anthropic)

- Safety baked into weights at training time
- Why it works for general safety: Prevents the worst harms at the model level
- Why it DOESN'T work for policy enforcement: Red team tests show model-level safety is a layer, not a boundary. Production systems need external enforcement.

2.3 The Key Insight From Industry

Every production-grade enforcement system separates into two components:

+-----+	+-----+
Policy Decision Point	Policy Administration Point
(PDP)	(PAP)
- Evaluates allow/deny	- CRUD on policy rules
- Local, fast (<5ms)	- Human-only access
- Read-only policy view	- Pushes updates to PDP
- Different trust domain	- Audit trail
from the agent	- Credential management
+-----+	+-----+

The agent can call the PDP (get decisions) but never the PAP (change policy).

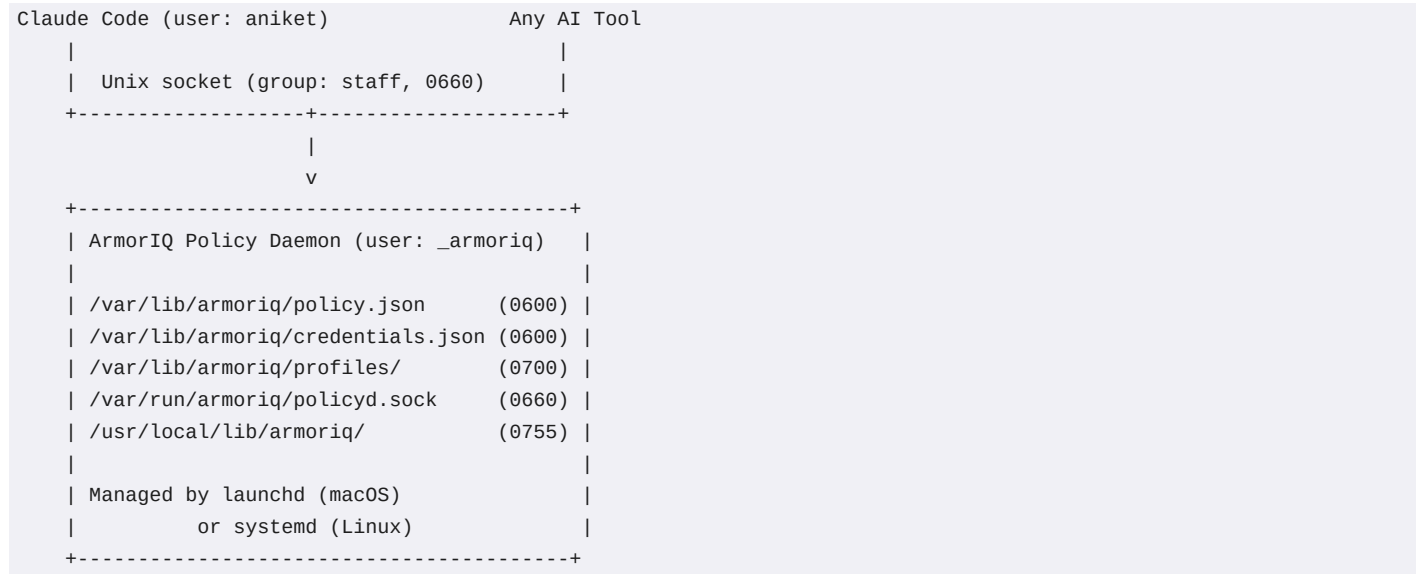
This is the OPAL/Cerbos/OPA model. And it's exactly what we need.

Part 3: What Will Work, What Won't

3.1 Approaches That WILL Work

A. System Daemon as Separate OS User (RECOMMENDED)

Run the policy daemon as a dedicated system user (`_armoriq`). Policy files, credentials, and daemon code are owned by this user. The AI agent's user cannot read/write them.



Why this works:

Security property	Mechanism	Bypassable?
LLM can't write policy files	Unix file permissions (owner <code>_armoriq</code> , mode <code>0600</code>)	No -- requires root or <code>_armoriq</code> user
LLM can't read credentials	Same -- owned by <code>_armoriq</code> , mode <code>0600</code>	No
LLM can't kill daemon	kill requires same user or root	No -- different user, launchd restarts it
LLM can't modify daemon code	Code owned by <code>_armoriq</code> under <code>/usr/local/lib/</code>	No
LLM can't send fake mutations via socket	Mutations require auth: 'user-session' tag only set by <code>UserPromptSubmit</code> hook	No -- LLM can't trigger <code>UserPromptSubmit</code>
LLM can't escalate to root	Standard non-root user	No -- unless kernel exploit (out of scope)

Why this is the right model:

- Same pattern as OPA sidecar, Cerbos PDP, Permit.io OPAL
- Uses OS-level primitives that are battle-tested for decades
- Sub-5ms latency (Unix socket, local evaluation)
- Works offline (no cloud dependency)
- Daemon already exists (`daemon.mjs`) -- 80% of code is written
- One daemon serves all ArmorIQ products

B. Cloud-Only Enforcement (enterprise add-on, not primary)

All policy decisions come from the ArmorIQ cloud backend. No local policy state.

Why this can't be the primary mode:

- 50-200ms per decision -- unacceptable for CLI tools
- No offline mode
- LLM could read the API key and make its own API calls
- Single point of failure

Verdict: Good as an enterprise add-on (org-wide policy sync, audit), bad as primary path.

3.2 Approaches That WON'T Work

C. More In-Process Guards (DOESN'T WORK)

Adding more regex checks, handler name blocking, or file monitoring within the same OS user is an arms race we can't win:

- Regex guards: Claude can base64-encode, use string concatenation, eval, or dynamic imports
- File watchers: Claude can kill the watcher (same user) or modify faster than watcher reacts
- Environment variable hiding: Claude has env, printenv, /proc/self/environ
- In-memory-only state: Claude can gcore the process or read /proc/<pid>/mem
- Encryption at rest: The decryption key must be accessible to the same process

Every guard you can build, the LLM can theoretically unbuild. It's the same trust domain.

D. Docker/Container Isolation (OVERKILL for CLI)

- Requires Docker installed -- heavyweight for a CLI plugin
- macOS Docker runs in a Linux VM (network bridging, volume mount complexity)
- Latency overhead from container networking

Verdict: Overkill. A system user + Unix socket achieves the same isolation with zero overhead.

E. Kernel-Level Enforcement (LSM, eBPF) (TOO COMPLEX)

- macOS doesn't have LSM/eBPF
- Requires root and kernel knowledge
- Extremely complex to configure correctly; fragile across OS updates

Verdict: Theoretically strongest, practically impossible for cross-platform CLI tool.

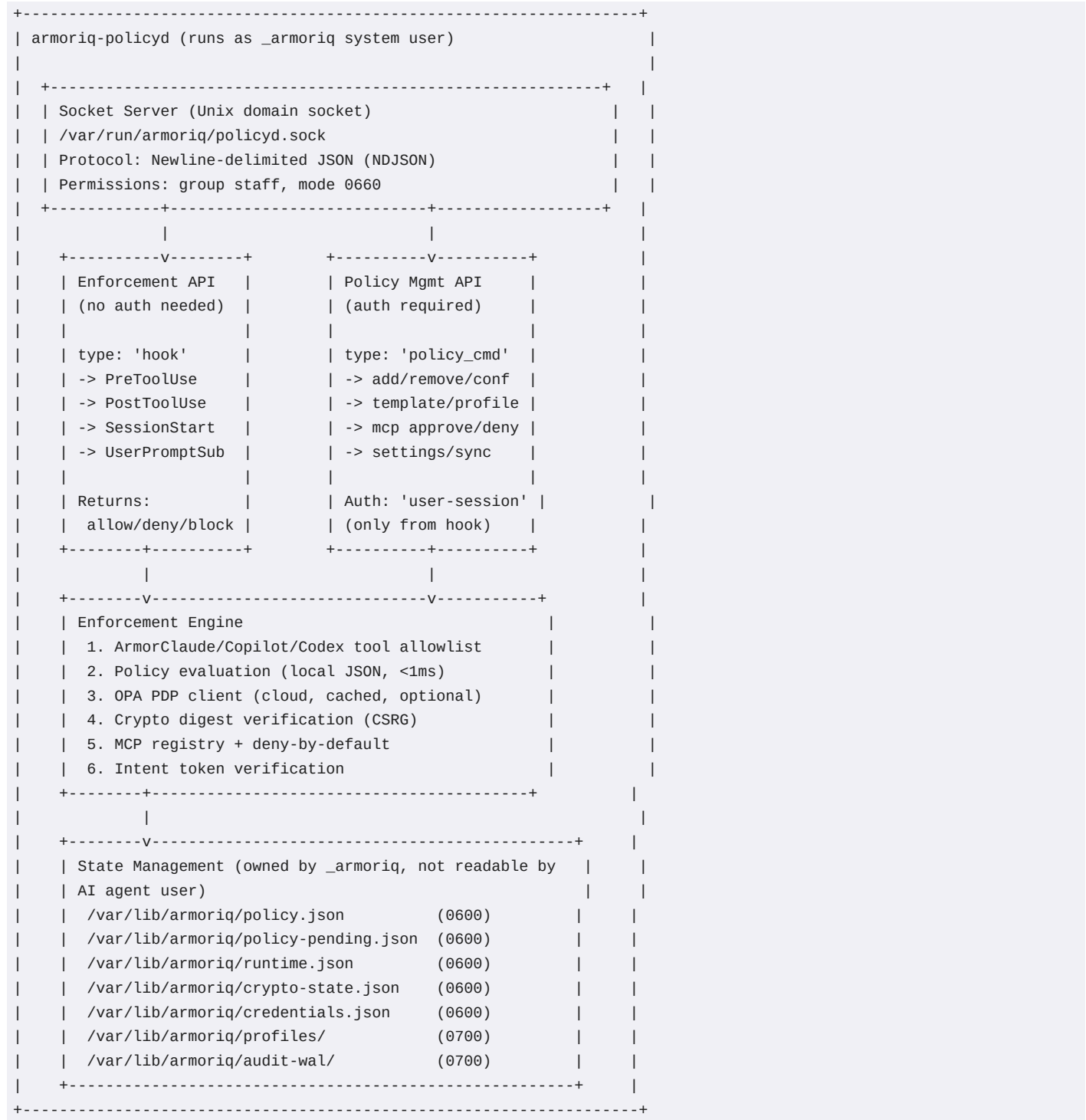
F. Trusted Execution Environment / Secure Enclave (OVERKILL)

- Enclave development is extremely complex; platform-specific
- Designed for protecting cryptographic keys, not for policy evaluation

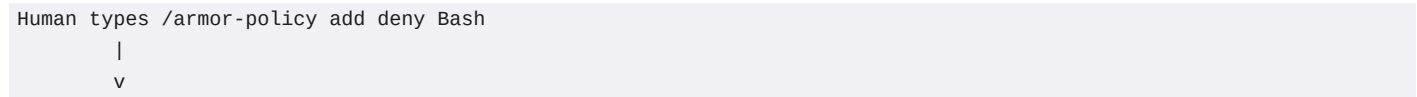
Verdict: Interesting for future credential storage, not for policy enforcement.

Part 4: Detailed Architecture

4.1 The ArmorIQ Policy Daemon (armoriq-policyd)



4.2 How Policy Mutation Works (Secure Flow)



```

Claude Code: registered skill? -> YES (stub)
      |
      v
UserPromptSubmit hook fires (Claude Code guarantee: human-only)
      |
      v
Hook process (runs as aniket user):
  1. Connects to /var/run/armoriq/policyd.sock
  2. Sends: {type:'policy_cmd', command:'/armor-policy add deny Bash',
            auth:'user-session', reqId:'r-1'}
  3. Daemon validates auth tag (only UserPromptSubmit sets this)
  4. Daemon stages change to policy-pending.json (it owns the file)
  5. Returns proposal text
  6. Hook calls blockPrompt(output) -> Claude LLM never sees this

```

Why Claude can NEVER do this:

1. Claude can't type in the terminal -> can't trigger UserPromptSubmit
2. Claude can't set auth: 'user-session' -> daemon rejects mutation without it
3. Even if Claude connects to socket directly, it can only send enforcement queries (no auth)
4. Claude can't write to /var/lib/armoriq/policy.json -> wrong user, permission denied
5. Claude can't kill the daemon -> wrong user, kill fails
6. Claude can't modify daemon code -> wrong user, files are read-only

4.3 How Enforcement Works (Fast Path)

```

Claude wants to call Bash('rm -rf /')
      |
      v
Claude Code fires PreToolUse hook
      |
      v
Hook process (runs as aniket user):
  1. Connects to /var/run/armoriq/policyd.sock
  2. Sends: {type:'hook', event:'PreToolUse', product:'armorclaude',
            input:{tool_name:'Bash', tool_input:{command:'rm -rf /'}}}
  3. Daemon evaluates policy (local JSON, <1ms)
  4. Returns: {permissionDecision:'deny',
              reason:'ArmorClaude: Bash is denied by policy'}
  5. Hook relays to Claude Code -> tool call blocked

Latency: <5ms total (socket connect + JSON + eval + response)

```

4.4 What ArmorClaude Looks Like After Migration

ArmorClaude shrinks from ~2000 lines of enforcement logic to ~200 lines of client code:

```

armorclaude/
+-- hooks/hooks.json           # Hook declarations (unchanged)
+-- skills/armor-policy/SKILL.md # Stub for /armor-policy routing
+-- .claude-plugin/plugin.json  # Plugin metadata
+-- scripts/
|   +-- bootstrap.mjs          # Dep installer + dispatcher
|   +-- hook-router.mjs        # THIN CLIENT: socket -> daemon
|   +-- policy-mcp.mjs         # MCP server (reads via daemon)
|   +-- lib/

```

```
|      +-- daemon-client.mjs      # Socket connect + NDJSON
+-- package.json
```

Everything else (engine, policy eval, templates, profiles, OPA, crypto, backend sync, audit) moves to armoriq-policyd.

4.5 Cross-Product Architecture



Part 5: Security Deep Dive

5.1 Attack Vector Analysis (Current vs Daemon)

#	Attack	Current (same user)	With daemon (separate user)
1	Claude writes to policy.json via Edit/Write	Path guard blocks (regex, bypassable)	Unix permissions block. Period.
2	Claude writes to policy.json via Bash redirect	WRITE_OPS regex blocks (bypassable)	Unix permissions block. Period.
3	Claude imports handler via Bash	Handler name regex (bypassable)	Handler code owned by _armoriq, can't import
4	Claude base64-encodes policy-modifying script	No guard catches this	Decoded script still can't write to _armoriq files
5	Claude kills the enforcement daemon	Same user, kill works	Different user, kill returns EPERM
6	Claude reads API key from credentials	Same user, cat works	File owned by _armoriq (0600), permission denied
7	Claude modifies daemon code/config	Same user, write works	Code owned by _armoriq, write denied
8	Claude sends policy mutation via socket	Can connect and send anything	Mutation requires auth: 'user-session' (can't set)
9	Claude chmod policy files	Same user, chmod works	Can't chmod files owned by different user
10	Claude uses sudo to escalate	Not available without password	Same -- not available without password
11	Prompt injection in tool output	/armor-policy text has no effect	Same -- still no effect
12	Claude social-engineers user	Human makes conscious decision	Same -- unavoidable

Score: Current = 4/12 blocked. Daemon = 11/12 blocked (only social engineering remains).

5.2 The Socket Authentication Model

The daemon socket is accessible to any user in the staff group (mode 0660). This means Claude CAN connect and send messages. Here's why that's still secure:

Read operations (no auth needed):

- type: 'hook' -> PreToolUse enforcement queries
- type: 'policy_read' -> list policies
- type: 'ping' -> health check

These are fine for Claude to call. Getting policy decisions is the whole point.

Write operations (auth required):

The auth: 'user-session' tag is set by the UserPromptSubmit hook handler. This hook only fires on human terminal input. Claude's process can connect to the socket, but it can't set auth: 'user-session' because:

1. The tag is hardcoded in hook-router.mjs inside the UserPromptSubmit handler

2. Claude can't modify hook-router.mjs (owned by plugin / _armoriq)
3. Even if Claude sends the tag directly, the daemon can verify via peer credentials

Defense in depth for socket auth:

- Level 1: auth tag presence (filters casual attempts)
- Level 2: Peer credential check (SO_PEERCREDS on Linux / LOCAL_PEERCREDS on macOS)
- Level 3: Verify PID is a child of Claude Code's hook spawn (process tree check)

5.3 Failure Modes

Scenario	Behavior	Why this is correct
Daemon not installed	ArmorClaude enters observe-only mode (audit, no enforcement)	User chose not to install; docs explain the trade-off
Daemon crashes	launchd/systemd restarts it; hooks fail-closed (deny-all) during gap	Better to block briefly than allow everything
Socket file deleted	Daemon recreates on restart; hooks fail-closed until socket exists	Same as crash -- transient, auto-recovers
Backend unreachable	Local enforcement continues; audit WAL queues until backend returns	Offline-first design
Policy file corrupted	Crypto digest mismatch -> daemon refuses to load -> deny-all + alert	Tamper detection at daemon level
OPA unreachable	Circuit breaker opens -> falls back to local JSON evaluation	Graceful degradation

5.4 Comparison: ArmorIQ vs Industry

Capability	OPA/Styra	Cerbos	Permit.io	Invariant	ArmorIQ (proposed)
Separate trust domain	Sidecar container	Sidecar/service	Sidecar PDP	Gateway proxy	System user daemon
Policy push mechanism	GCS bundles	Git/S3 sync	OPAL pub/sub	Gateway config	Backend sync + local CRUD
Evaluation latency	Sub-ms (local)	Sub-ms (local)	1-5ms (local)	Near-zero	<5ms (Unix socket)
Offline support	Yes (cached bundles)	Yes (local policy)	Partial (cached)	No (needs proxy)	Yes (local JSON)
AI agent awareness	No	No	No	Yes (built for AI)	Yes (built for AI)
Multi-product	Generic	Generic	Generic	AI agents only	AI tools specific
Human-only mutation	N/A (ops workflow)	N/A (git PR)	N/A (dashboard)	N/A	UserPromptSubmit hook
Crypto tamper detection	Bundle signing	No	No	No	CSRG Merkle proofs

ArmorIQ's unique differentiators:

1. Purpose-built for AI tool enforcement (not generic authz)
2. Human-only policy mutation via Claude Code's UserPromptSubmit architectural guarantee

- 3. Cryptographic tamper detection via CSRG tokens with Merkle proofs
- 4. Multi-AI-tool support from a single daemon (Claude, Copilot, Codex)
- 5. Offline-first with optional cloud sync (vs cloud-dependent competitors)

Part 6: Implementation Strategy

6.1 Phase Overview

Phase	What	Effort	Delivers
Phase 0	Ship current architecture (same-user) as v0.3	1 week	Working policy system, 195 tests, immediate value
Phase 1	Create armoriq-policyd repo, extract daemon	2 weeks	Standalone daemon with socket API
Phase 2	System service installer (launchd + systemd)	2 weeks	OS-level isolation, _armoriq user
Phase 3	ArmorClaude thin client migration	1 week	ArmorClaude becomes ~200 lines
Phase 4	Socket authentication hardening	1 week	Peer credential checks, nonce verification
Phase 5	Cross-product client library	2 weeks	@armoriq/policy-client npm + Python packages

6.2 What Goes Where

armoriq-policyd (new repo) -- the daemon and all enforcement logic:

- daemon.mjs (socket server, message routing)
- engine.mjs (enforcement handlers)
- armor-policy-commands.mjs (policy CRUD)
- policy.mjs, policy-templates.mjs, policy-profiles.mjs
- tool-registry.mjs, opa-client.mjs, policy-compiler.mjs
- crypto-policy.mjs, backend-client.mjs
- runtime-state.mjs, audit-wal.mjs, fs-store.mjs
- config.mjs (daemon-specific, reads from /var/lib/armoriq/)
- install/ (installer scripts, launchd plist, systemd unit)

armorclaudes (existing repo) -- thin client only:

- hook-router.mjs (connect to daemon, relay events)
- daemon-client.mjs (socket connect + NDJSON)
- policy-mcp.mjs (MCP server, reads via daemon)
- skills/armor-policy/SKILL.md (routing stub)

6.3 Installer Experience

```
# Developer runs:
curl -fsSL https://install.armoriq.ai/policyd | sudo bash
```

```
# What happens:
# 1. Creates _armoriq system user (if not exists)
# 2. Creates /var/lib/armoriq/ with correct ownership
# 3. Installs daemon to /usr/local/lib/armoriq/
# 4. Installs launchd plist (macOS) or systemd unit (Linux)
# 5. Starts the daemon
# 6. Adds current user to staff group (for socket access)
# 7. Prints: 'ArmorIQ Policy Daemon installed.'

# Verify:
armoriq-policyd status
# -> ArmorIQ Policy Daemon v1.0.0 | running | uptime: 5s
```

Part 7: Open Questions for Brainstorming

1. Should the daemon support Windows?

Named pipes instead of Unix sockets, Windows service instead of launchd/systemd

2. Should we build an admin dashboard?

Web UI for policy management, complementing /armor-policy CLI commands

3. Should the daemon expose metrics?

Prometheus endpoint for decision counts, latency percentiles, deny rates

4. How should org-wide policy distribution work?

Backend pushes to all daemons? Daemons poll? Git-based like Cerbos?

5. Should we support 'break glass'?

Emergency override that bypasses all policy -- requires physical MFA token?

6. Homebrew formula?

brew install armoriq-policyd for easier macOS installation, avoids curl-pipe-sudo

7. How does this interact with Claude Code's own permission system?

Claude Code has its own allow/deny for tools -- should ArmorIQ layer on top, or replace it?

Part 8: Risk Assessment

Risk	Impact	Likeliho	Mitigation
sudo installation friction reduces adoption	High	Medium	Homebrew formula; clear docs; observe-only mode without daemon
macOS Gatekeeper blocks unsigned daemon	Medium	High	Sign with Apple Developer ID or distribute via Homebrew
Socket permission misconfiguration	High	Low	Installer validates; daemon refuses to start if insecure
Users lose policy when daemon crashes	Medium	Low	launchd/systemd auto-restart; WAL for audit durability
Version mismatch (client vs daemon)	Medium	Medium	Version handshake in ping; client warns on mismatch
Breaking change in Claude Code hook system	High	Low	Hooks are Claude Code's documented public API; unlikely to break
Competitor ships similar product faster	Medium	Medium	Ship v0.3 (current) immediately; daemon is the v1.0 upgrade